Methods 8

John Dong

Apologies for the late submission! You can view my repo's commit history here if that helps to verify that this is my own work done independent from the solutions.

1.

a.

$$\text{Provided that :} \qquad \bar{Y}(A=0,B=0)=1, \qquad \bar{Y}(A=1,B=0)=5,$$

$$\bar{Y}(A=0,B=1)=4, \qquad \bar{Y}(A=1,B=1)=7$$

we can hold the following to be true.

$$\begin{aligned}\bar{Y}(A=0,B=0) &= \mathbb{E}(Y_i \mid A=0,B=0)\\ &= \mathbb{E}(\beta_0+\beta_1\cdot 0+\beta_2\cdot 0+\beta_3\cdot 0+\epsilon_i)\\ &= \beta_0+0+0+0+\mathbb{E}(\epsilon_i)\\ &= \beta_0+0=1\\ &\therefore \beta_0=1\end{aligned}$$

$$\begin{aligned}\bar{Y}(A=1,B=0) &= \mathbb{E}(Y_i \mid A=1,B=0)\\ &= \mathbb{E}(\beta_0+\beta_1\cdot 1+\beta_2\cdot 0+\beta_3\cdot 0+\epsilon_i)\\ &= \beta_0+\beta_1+0+0+\mathbb{E}(\epsilon_i)\\ &= 1+\beta_1+0=5\\ &\therefore \beta_1=4\end{aligned}$$

$$\begin{aligned}
\bar{Y}(A=0,B=1) &= \mathbb{E}(Y_i \mid A=0,B=1) \\
&= \mathbb{E}(\beta_0 + \beta_1 \cdot 0 + \beta_2 \cdot 1 + \beta_3 \cdot 0 + \epsilon_i) \\
&= \beta_0 + 0 + \beta_2 + 0 + \mathbb{E}(\epsilon_i) \\
&= 1 + \beta_2 + 0 = 4 \\
&\therefore \beta_2 = 3
\end{aligned}$$

$$\begin{aligned}
\bar{Y}(A=1,B=1) &= \mathbb{E}(Y_i \mid A=1,B=1) \\
&= \mathbb{E}(\beta_0 + \beta_1 \cdot 1 + \beta_2 \cdot 1 + \beta_3 \cdot 1 + \epsilon_i) \\
&= \beta_0 + \beta_1 + \beta_2 + \beta_3 + \mathbb{E}(\epsilon_i) \\
&= 1 + 4 + 3 + \beta_3 = 7 \\
&\therefore \beta_3 = -1
\end{aligned}$$

Final Answer: $\boxed{\beta_0 = 1, \beta_1 = 4, \beta_2 = 3, \beta_3 = -1}$

b.

In order to get estimates for the four regression coefficients, we need estimates for the four averages above. We have two variables, A and B, which can take values 0 or 1 depending on whether or not the value observed was at a "high" or "low" level for each. β_0, the intercept or "baseline", necessitates that we have a treatment group where $A = 0, B = 0$. Then, by comparing to our baseline, we can get our treatment effects for A and B by changing the level one at a time: one group is $A = 1, B = 0$, and the other $A = 0, B = 1$. This lets us estimate β_1 and β_2. We also have to estimate the interaction effect β_3, so we need treatment group $A = 1, B = 1$. This constructs our full factorial design, where we have 2^2 treatment groups, one for each combination of treatment levels. Then, our observational units are randomly assigned to evenly split among the treatment groups.

In summary:

Group	Treatment	Number of units
Group 1	$A = 0, B = 0$	$n = 50$
Group 2	$A = 1, B = 0$	$n = 50$
Group 3	$A = 0, B = 1$	$n = 50$
Group 4	$A = 1, B = 1$	$n = 50$

c.

A full factorial design would be better:

We need to compare $A = 0,\ B = 0,\ C = 0$ and $A = 0,\ B = 0,\ C = 1$ to see C's treatment effect; i.e. how it changes the predicted result when all other variables are held constant. Then, to see how the effect changes for A and B, we need to compare $A = 0,\ B = 0,\ C = 1$ versus $A = 1,\ B = 0,\ C = 1$; likewise for B, we need to compare \$ A = 0, ~B = 0, ~C = 1\$ to $A = 0,\ B = 1,\ C = 1$. Assuming that our treatment groups from **(b)** had level $C = 0$, then this introduces 3 new groups. That gives us 7 treatment groups to estimate the interaction effects described.

A full factorial design would also include a three-way interaction effect, getting us $7 + 1 = 2^3$ groups, each group getting 25 units through random assignment. While a three-way interaction effect was not mentioned in the prompt, we also don't have reasonable certainty to assume that interaction effect to be zero. So, if we did a fractional factorial design, we would be makiing the dubious assumption that there is no three-way interaction effect, and we would also be missing the interactions between C and A or B, or we would be missing treatment effects.

d.

To consider treatment and interaction effects with C, we assume:

$$Y_i = \beta_0 + \beta_1 A_i + \beta_2 B_i + \beta_3 AB_i + \beta_4 C_i + \beta_5 AC_i + \beta_6 BC_i + \beta_7 ABC_i + \epsilon_i$$

With the assistant's proposed fractional factorial design, we can estimate $\bar{Y}(A = 1, B = 0, C = 0)$, $\bar{Y}(A = 0, B = 1, C = 0)$, $\bar{Y}(A = 0, B = 0, C = 1)$, and $\bar{Y}(A = 1, B = 1, C = 1)$ to estimate β_1, β_2, β_4, and β_7.

However, recall that the population averages gave us β_0, β_1 (treatment effect of a), β_2 (treatment effect of b), and β_3 (interaction effect between A and B). Therefore, in this fractional design, the interaction effect between A and B is aliased with the treatment effect of C. Therefore, we can't estimate the treatment effect of C.

e.

The design from part **(d)** assumes that the regression coefficients that it can't estimate (the experiment results can't estimate $\beta_0, \beta_3, \beta_5, \beta_6$) to be 0. This includes the interaction effect between A and B, β_3. So, if we assume $\beta_3 = 0$:

$$\begin{aligned}\bar{Y}(A=1,B=1) &= \beta_0 + \beta_1 + \beta_2 + \beta_3 \\ &= 1 + 4 + 3 + 0 \\ &= 8\end{aligned}$$

Therefore, to estimate treatment effect of C, we would require $\bar{Y}(A=1,B=1) = 8$.

2.

Run `FourFactorAnalysis.R`:

```
source("FourFactorAnalysis.R")
```

a.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.1     v tibble    3.2.1
v lubridate 1.9.3     v tidyr     1.3.1
v purrr     1.0.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(kableExtra)
```

```
Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

    group_rows
```

```r
pairwise_lm = lm(y ~ a + b + c + d + a*b + a*c + a*d +
                   b*c + b*d + c*d, data = thedata)

coeff_pop = c("Intercept" = trt0, "A" = trta, "B" = trtb, "C" = trtc, "D" = trtd, "AB" = trta
tibble(
  coefficient = names(coeff_pop)[-(12:16)],
  estimate = unname(pairwise_lm$coefficients),
  pop_value = unname(coeff_pop)[-(12:16)]
) |> kable(digits = 3)
```

coefficient	estimate	pop_value
Intercept	10.099	10
A	1.948	2
B	-1.055	-1
C	3.825	4
D	3.028	3
AB	3.985	4
AC	-0.964	-1
AD	2.023	2
BC	6.327	6
BD	-3.114	-3
CD	1.979	2

Looking at the outputs for the regression and for the true coefficients side by side, we can see that they are very similar (tend to only be off by $\approx \pm 0.3$ at most).

b.

Our coefficient for the interaction effect between B and C is ≈ 6.102. We also have expected coefficients for just B and just C. So, if an individual gets treatments B and C and we have **no** interaction effect, the result is expected to change by the amounts denoted in the treatment coefficients for B and C, holding all else constant. However, with an interaction effect, the results for an individual who gets both treatments B and C is expected to change by an additional $+6.102$. Note that this interaction effect is zero if an individual doesn't get treatment B and/or doesn't get treatment C; it only is nonzero for an individual who recieved both treatments.

c.

```
lm_3c = lm(y ~ a + b + c + d + a*b + a*c + a*d + b*c + b*d +
             c*d + a*b*c + b*c*d + a*c*d + a*b*d, data = thedata)

tibble(
  coefficient = names(coeff_pop)[-16],
  estimate = unname(lm_3c$coefficients),
  p_value = summary(lm_3c)$coefficients[, "Pr(>|t|)"] |> unname(),
  pop_value = unname(coeff_pop)[-16]
) |> kable(digits = 3)
```

coefficient	estimate	p_value	pop_value
Intercept	10.050	0.000	10
A	1.947	0.000	2
B	-0.900	0.000	-1
C	3.931	0.000	4
D	3.065	0.000	3
AB	3.872	0.000	4
AC	-0.976	0.000	-1
AD	2.149	0.000	2
BC	6.003	0.000	6
BD	-3.301	0.000	-3
CD	1.893	0.000	2
ABC	0.250	0.177	0
BCD	0.398	0.034	0
ACD	-0.226	0.221	0
ABD	-0.025	0.892	0

```
#nested f test

anova(pairwise_lm, lm_3c)
```

```
Analysis of Variance Table

Model 1: y ~ a + b + c + d + a * b + a * c + a * d + b * c + b * d + c *
    d
Model 2: y ~ a + b + c + d + a * b + a * c + a * d + b * c + b * d + c *
    d + a * b * c + b * c * d + a * c * d + a * b * d
```

```
  Res.Df    RSS Df Sum of Sq      F Pr(>F)
1     53 1.9063
2     49 1.6333  4   0.27299 2.0475 0.1022
```

Both the main effect and pairwise interaction coefficients are similar to the true values in both models. We get a high p-value from the test, meaning our observed three-way coefficients wouldn't too atypical assuming the true three-way coefficients were 0, which they are. This means we fail to reject the F-test's null hypothesis.

3.

a.

Note that the assignments for the half replicate either have:

- Low level for all treatments
- Low level for exactly 2 treatments, high level for exactly 2 treatments
- High level for all treatments

And each assignment that satisfies one of these conditions is precisely then included in the half replicate

```
half_data = thedata |>
  filter(
    a + b + c + d == 0 | a + b + c + d == 2 | a + b + c + d == 4
  ) |> remove_rownames()

half_data |> kable(digits = 3)
```

a	b	c	d	y
0	0	0	0	10.445
0	0	0	0	9.776
0	0	0	0	10.119
0	0	0	0	9.913
1	1	0	0	15.091
1	1	0	0	15.129
1	1	0	0	15.091
1	1	0	0	14.622

a	b	c	d	y
1	0	1	0	15.264
1	0	1	0	15.002
1	0	1	0	14.493
1	0	1	0	15.100
1	0	0	1	17.133
1	0	0	1	17.137
1	0	0	1	17.258
1	0	0	1	17.369
0	1	1	0	19.112
0	1	1	0	19.210
0	1	1	0	19.042
0	1	1	0	19.023
0	1	0	1	8.942
0	1	0	1	8.858
0	1	0	1	8.960
0	1	0	1	8.950
0	0	1	1	19.169
0	0	1	1	18.764
0	0	1	1	18.690
0	0	1	1	19.184
1	1	1	1	28.257
1	1	1	1	28.165
1	1	1	1	28.020
1	1	1	1	28.129

b.

```
bad_pairs = lm(y ~ a + b + c + d + a*b + a*c + a*d + b*c + b*d + c*d ,data = half_data)
summary(bad_pairs)
```

```
Call:
lm(formula = y ~ a + b + c + d + a * b + a * c + a * d + b *
    c + b * d + c * d, data = half_data)

Residuals:
     Min       1Q   Median       3Q      Max
-0.47166 -0.08859  0.02229  0.11329  0.38208
```

```
Coefficients: (3 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  10.0631     0.1051  95.748  < 2e-16 ***
a            -0.5484     0.1486  -3.690  0.00115 **
b            -0.4951     0.1051  -4.711 8.66e-05 ***
c             9.5290     0.1051  90.666  < 2e-16 ***
d            -0.6404     0.1051  -6.094 2.71e-06 ***
a:b           5.9636     0.1486  40.123  < 2e-16 ***
a:c          -4.0788     0.1486 -27.442  < 2e-16 ***
a:d           8.3499     0.1486  56.177  < 2e-16 ***
b:c               NA         NA      NA       NA
b:d               NA         NA      NA       NA
c:d               NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2102 on 24 degrees of freedom
Multiple R-squared:  0.999, Adjusted R-squared:  0.9986
F-statistic:  3262 on 7 and 24 DF,  p-value: < 2.2e-16
```

We get NAs for three of the coefficient estimates for pairwise interaction effects. Therefore, R only allows us to have three pairwise interaction coefficient estimates at a time.

c.

```r
pairs_c = lm(y ~ a + b + c + d + a*b + c*d ,data = half_data)
summary(pairs_c)
```

```
Call:
lm(formula = y ~ a + b + c + d + a * b + c * d, data = half_data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.5788 -1.7736 -0.0278  1.8413  3.2522

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   8.9954     1.1195   8.036 1.63e-08 ***
```

```
a             1.5871     1.2926   1.228 0.230514
b            -0.4951     1.2926  -0.383 0.704819
c             7.4896     0.9140   8.194 1.13e-08 ***
d             3.5345     0.9140   3.867 0.000661 ***
a:b           5.9636     1.8281   3.262 0.003086 **
c:d               NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.585 on 26 degrees of freedom
Multiple R-squared:  0.828, Adjusted R-squared:  0.7949
F-statistic: 25.03 on 5 and 26 DF,  p-value: 3.604e-09
```

No! R gives a coefficient estimate for only one of the pairwise interactions.

d.

```
pairs_d = lm(y ~ a + b + c + d + a*b + a*c + a*d ,data = half_data)
summary(pairs_d)$coefficients |> bind_cols(tibble(true_val = coeff_pop[1:8])) |> kable(digits
```

Estimate	Std. Error	t value	Pr(>|t|)	true_val
10.063	0.105	95.748	0.000	10
-0.548	0.149	-3.690	0.001	2
-0.495	0.105	-4.711	0.000	-1
9.529	0.105	90.666	0.000	4
-0.640	0.105	-6.094	0.000	3
5.964	0.149	40.123	0.000	4
-4.079	0.149	-27.442	0.000	-1
8.350	0.149	56.177	0.000	2

The estimates do not match the true coefficients.

4.

a.

```r
#this script is about analyzing full factorial designs
#we will simulate data from a 2^4 full factorial design

#set up a true mean value for cases that get no treatments at all
trt0 = 10

#set up true values for the main effects for 4 treatments
trta = 2
trtb = -1
trtc = 4
trtd = 3


#set up true values for the effects for two-way interactions
trtab = 4
trtac = -1
trtad = 2
trtbc = 0
trtbd = 0
trtcd = 0

#we will set all three-way interactions and the four-way interaction to zero
trtabc = 0
trtabd = 0
trtacd = 0
trtbcd = 0
trtabcd = 0


#let's write down all 16 treatments as combinations of 1s and 0s
#the first column is for treatment a, the second for treatment b, the third for treatment c,
#we use a 1 when the treatment is active and a 0 when it is not

x0 = c(0, 0, 0, 0)
xa = c(1, 0, 0, 0)
xb = c(0, 1, 0, 0)
xc = c(0, 0, 1, 0)
xd = c(0, 0, 0, 1)
xab = c(1, 1, 0, 0)
xac = c(1, 0, 1, 0)
xad = c(1, 0, 0, 1)
xbc = c(0, 1, 1, 0)
```

```
xbd = c(0, 1, 0, 1)
xcd = c(0, 0, 1, 1)
xabc = c(1, 1, 1, 0)
xabd = c(1, 1, 0, 1)
xacd = c(1, 0, 1, 1)
xbcd = c(0, 1, 1, 1)
xabcd = c(1, 1, 1, 1)

## let's make some datasets with 4 observations per treatment assignment

n = 4
sigma = 0.2
y0 = rnorm(n, trt0, sigma)
ya = rnorm(n, trt0 + trta, sigma)
yb = rnorm(n, trt0 + trtb, sigma)
yc = rnorm(n, trt0 + trtc, sigma)
yd = rnorm(n, trt0 + trtd, sigma)
yab = rnorm(n, trt0 + trta + trtb + trtab, sigma)
yac = rnorm(n, trt0 + trta + trtc + trtac, sigma)
yad = rnorm(n, trt0 + trta + trtd + trtad, sigma)
ybc = rnorm(n, trt0 + trtb + trtc + trtbc, sigma)
ybd = rnorm(n, trt0 + trtb + trtd + trtbd, sigma)
ycd = rnorm(n, trt0 + trtc + trtd + trtcd, sigma)
yabc = rnorm(n, trt0 + trta + trtb + trtc + trtab + trtbc + trtac + trtabc, sigma)
yabd = rnorm(n, trt0 + trta + trtb + trtd + trtab + trtad + trtbd + trtabd, sigma)
yacd = rnorm(n, trt0 + trta + trtc + trtd + trtac + trtad + trtcd + trtacd, sigma)
ybcd = rnorm(n, trt0 + trtb + trtc + trtd + trtbc + trtbd + trtcd + trtbcd, sigma)
yabcd = rnorm(n, trt0 + trta + trtb + trtc + trtd + trtab + trtac + trtad + trtbc + trtbd + t

#make a matrix with 4 rows per treatment group in order of treatment group
thedata = rbind(x0, x0, x0, x0)
thedata = rbind(thedata, xa, xa, xa, xa)
thedata = rbind(thedata, xb, xb, xb, xb)
thedata = rbind(thedata, xc, xc, xc, xc)
thedata = rbind(thedata, xd, xd, xd, xd)
thedata = rbind(thedata, xab, xab, xab, xab)
thedata = rbind(thedata, xac, xac, xac, xac)
thedata = rbind(thedata, xad, xad, xad, xad)
thedata = rbind(thedata, xbc, xbc, xbc, xbc)
thedata = rbind(thedata, xbd, xbd, xbd, xbd)
thedata = rbind(thedata, xcd, xcd, xcd, xcd)
thedata = rbind(thedata, xabc, xabc, xabc, xabc)
```

```
thedata = rbind(thedata, xabd, xabd, xabd, xabd)
thedata = rbind(thedata, xacd, xacd, xacd, xacd)
thedata = rbind(thedata, xbcd, xbcd, xbcd, xbcd)
thedata = rbind(thedata, xabcd, xabcd, xabcd, xabcd)

#now add the y column, which is in the same order as thedata
y = c(y0, ya, yb, yc, yd, yab, yac, yad, ybc, ybd, ycd, yabc, yabd, yacd, ybcd, yabcd)
thedata = cbind(thedata, y)
thedata = data.frame(thedata)
names(thedata)[1:4] = c("a", "b", "c", "d")

half_data = thedata |>
  filter(
    a + b + c + d == 0 | a + b + c + d == 2 | a + b + c + d == 4
  ) |> remove_rownames()

half_data |> kable(digits = 3)
```

a	b	c	d	y
0	0	0	0	10.009
0	0	0	0	10.195
0	0	0	0	9.872
0	0	0	0	9.868
1	1	0	0	14.855
1	1	0	0	14.805
1	1	0	0	15.144
1	1	0	0	15.397
1	0	1	0	15.181
1	0	1	0	15.161
1	0	1	0	14.928
1	0	1	0	14.753
1	0	0	1	17.015
1	0	0	1	17.050
1	0	0	1	16.734
1	0	0	1	17.065
0	1	1	0	13.204
0	1	1	0	12.814
0	1	1	0	13.000
0	1	1	0	12.875
0	1	0	1	11.850
0	1	0	1	11.978

a	b	c	d	y
0	1	0	1	12.075
0	1	0	1	11.977
0	0	1	1	16.994
0	0	1	1	17.150
0	0	1	1	17.237
0	0	1	1	16.937
1	1	1	1	23.139
1	1	1	1	22.884
1	1	1	1	23.036
1	1	1	1	23.200

b.

```
test_pairs4 = lm(y ~ a + b + c + d + a*b + a*c + a*d + b*c + b*d + c*d ,data = half_data)
summary(test_pairs4)
```

```
Call:
lm(formula = y ~ a + b + c + d + a * b + a * c + a * d + b *
    c + b * d + c * d, data = half_data)

Residuals:
     Min       1Q   Median       3Q      Max
-0.25296 -0.11840  0.01535  0.10049  0.34644

Coefficients: (3 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  9.98602    0.08689  114.92  < 2e-16 ***
a            1.99269    0.12288   16.22 1.97e-14 ***
b           -1.06110    0.08689  -12.21 8.70e-12 ***
c            4.04823    0.08689   46.59  < 2e-16 ***
d            3.04507    0.08689   35.04  < 2e-16 ***
a:b          4.13263    0.12288   33.63  < 2e-16 ***
a:c         -1.02113    0.12288   -8.31 1.60e-08 ***
a:d          1.94226    0.12288   15.81 3.46e-14 ***
b:c               NA         NA      NA       NA
b:d               NA         NA      NA       NA
c:d               NA         NA      NA       NA
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1738 on 24 degrees of freedom
Multiple R-squared:  0.9984,    Adjusted R-squared:  0.9979
F-statistic:  2102 on 7 and 24 DF,  p-value: < 2.2e-16
```

Again, the last three pairwise interaction estimates are NAs.

c.

```
fit_pairs4 = lm(y ~ a + b + c + d + a*b + a*c + a*d,data = half_data)
summary(fit_pairs4)$coefficients |>
  bind_cols(tibble(true_val = coeff_pop[1:8])) |>
  kable(digits = 3)
```

Estimate	Std. Error	t value	Pr(>|t|)	true_val
9.986	0.087	114.924	0	10
1.993	0.123	16.216	0	2
-1.061	0.087	-12.212	0	-1
4.048	0.087	46.589	0	4
3.045	0.087	35.044	0	3
4.133	0.123	33.630	0	4
-1.021	0.123	-8.310	0	-1
1.942	0.123	15.806	0	2

With a revised dataset dealing with aliasing between pairwise interaction effects, we now get accurate estimates for the pairwise coefficients in this half replicate design.